CÔNG NGHỆ PHẦN MỀM MỚI (MTSE431179)

Json Web Token

THS. NGUYỄN HỮU TRUNG

Thông tin Giảng viên

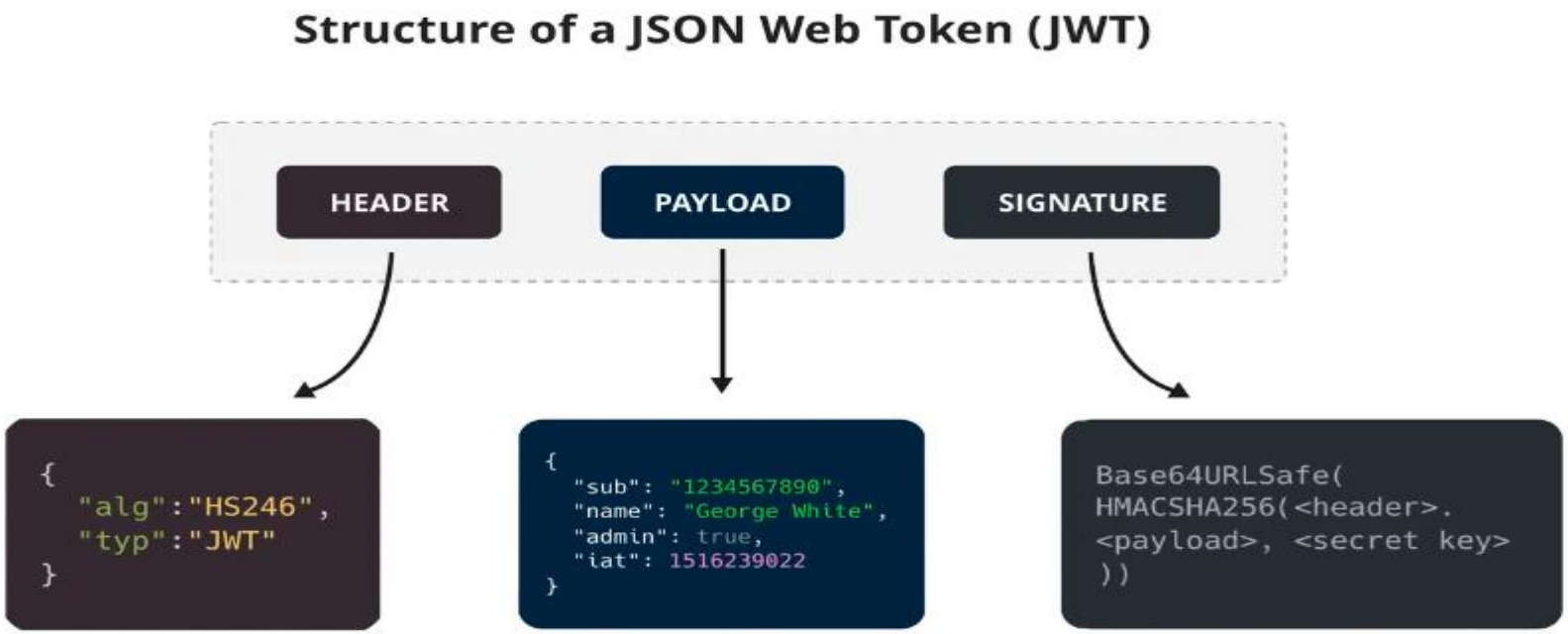
- Ths. Nguyễn Hữu Trung
- Khoa Công Nghệ Thông Tin
- Trường Đại học Sư Phạm Kỹ Thuật TP.HCM
- 090.861.7108
- trungnh@hcmute.edu.vn
- https://www.youtube.com/@baigiai

Nội dung

- Giới thiệu JWT
- Demo JWT trên ExpressJS

JWT là gì?

- **JWT** là một phương tiện đại diện cho các yêu cầu chuyển giao giữa hai bên Client – Server , các thông tin trong chuỗi **JWT** được định dạng bằng JSON. Trong đó chuỗi Token phải có 3 phần là **header** , phần **payload** và phần **signature** được ngăn bằng dấu "."
- JWT có thể được ký bằng cách sử dụng một secret (với thuật toán HMAC) hoặc cặp public/private key dùng chuẩn RSA hoặc ECDSA



Cấu trúc của JSON Web Token

- Phần **header** sẽ chứa kiểu dữ liệu và thuật toán sử dụng để mã hóa ra chuỗi JWT

{ "typ": "JWT", "alg": "HS256" }

- **"typ" (type)** chỉ ra rằng đối tượng là một **JWT**
- **"alg" (algorithm)** xác định thuật toán mã hóa cho chuỗi là **HS256**

- Phần **payload** sẽ chứa các thông tin mình muốn đặt trong chuỗi **Token** như **username** , **userId** , **author** , … ví dụ: { "user_name": "admin", "user_id": "1", "authorities": "ADMIN", "images": "u1.jpg" }

- *Signature* là phần cuối cùng của *JWT*, có chức năng xác thực danh tính người gửi. Phần chử ký này sẽ được tạo ra bằng cách mã hóa phần **header** , **payload** kèm theo một chuỗi **secret (khóa bí mật)** , ví dụ:

data = base64urlEncode(header) + "." + base64urlEncode(payload) signature = Hash(data, secret);

// header.payload.signature
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhdWQiOlsidGVzdGp3dHJlc291cmNlaWQiXSwidXNlcl9uYW1lIjoiYWRtaW4iLCJzY29wZSI6WyJyZWFkIiwid3JpdGUiXSwiZXhwIjoxNTEzNzE.
9nRhBWiRoryc8fV5xRpTmw9iyJ6EM7WTGTjvCM1e36Q

Cấu trúc của JSON Web Token

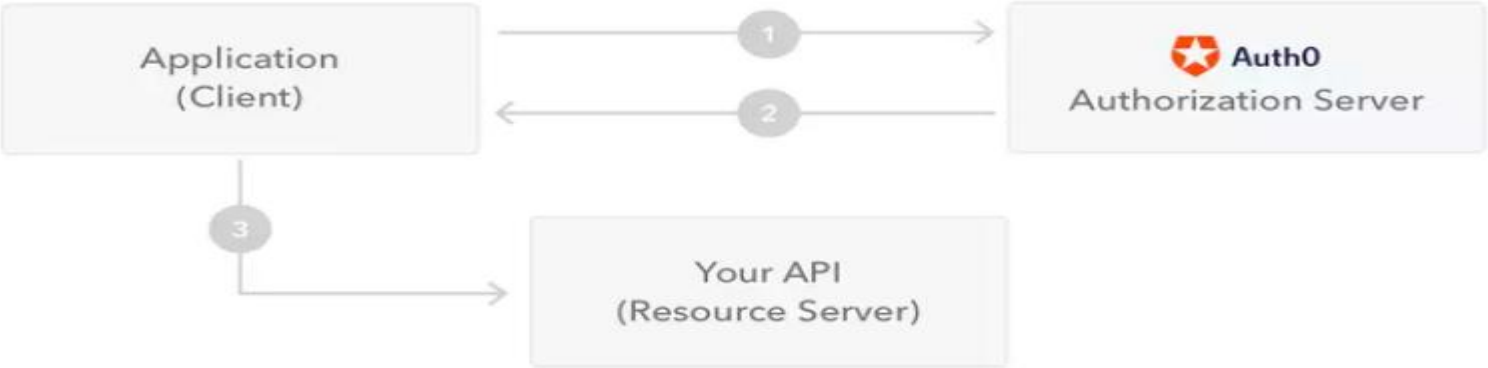
- *Payload* trong JWT chứa các *claims*. *Claims* là tập hợp các thông tin đại diện cho một thực thể (*object*) (ví dụ : user_id) và một số thông tin đi kèm. *Claims* sẽ có dạng *Key - Value*. Do đó, chúng ta có thể hiểu rằng, *claims* ám chỉ việc yêu cầu truy xuất tài nguyên cho *object* tương ứng.
- Có 3 kiểu *claims*, bao gồm *registered*, *public*, and *private claims*.
  - *Registered Claims* là các thành phần được xác định trước của *claims*. Thành phần này mặc dù không bắt buộc, nhưng là thành phần nên có để cung cấp một số chức năng và thông tin hữu ích. Một số *registered claims* bao gồm :
    - iss (issuer): Tổ chức, đơn vị cung cấp, phát hành *JWT*.
    - sub (subject): Chủ thể của *JWT*, xác định rằng đây là người sở hữu hoặc có quyền truy cập các *resource* (tài nguyên).
    - aud (audience): Được hiểu là người nhận thông tin, và có thể xác thực tính hợp lệ của *JWT*.
    - exp (expiration time): Thời hạn của *JWT*, vượt quá thời gian này, *JWT* được coi là không hợp lệ.

Cấu trúc của JSON Web Token

- Có 3 kiểu *claims*, bao gồm *registered*, *public*, and *private claims*.
  - *Public claims* là các thành phần được xác định bởi người sử dụng *JWT*, được sử dụng rộng rãi trong *JWT*. Mặc dù việc sử dụng *public claims* không phải là bắt buộc, tuy nhiên để tránh xảy ra xung đột. Một số *public claims* điển hình :
    - name, given_name, family_name, middle_name: Thông tin tên nói chung của user
    - email: Thông tin email của user.
    - locale : Địa chỉ của user.
    - profile, picture : Thông tin của trang web gửi đến.
- Các bên sử dụng *JWT* có thể sẽ cần sử dụng đến *claims* không phải là *Registered Claims,* cũng không được định nghĩa trước như *Public Claims.* Đây là phần thông tin mà các bên tự thoả thuận với nhau, không có tài liệu hay tiêu chuẩn nào dành cho *private claims.* **Lưu ý:** Tên *claims* chỉ nên dài ba ký tự vì *JWT* hướng tới sự nhỏ gọn.

JSON Web Tokens hoạt động như thế nào?

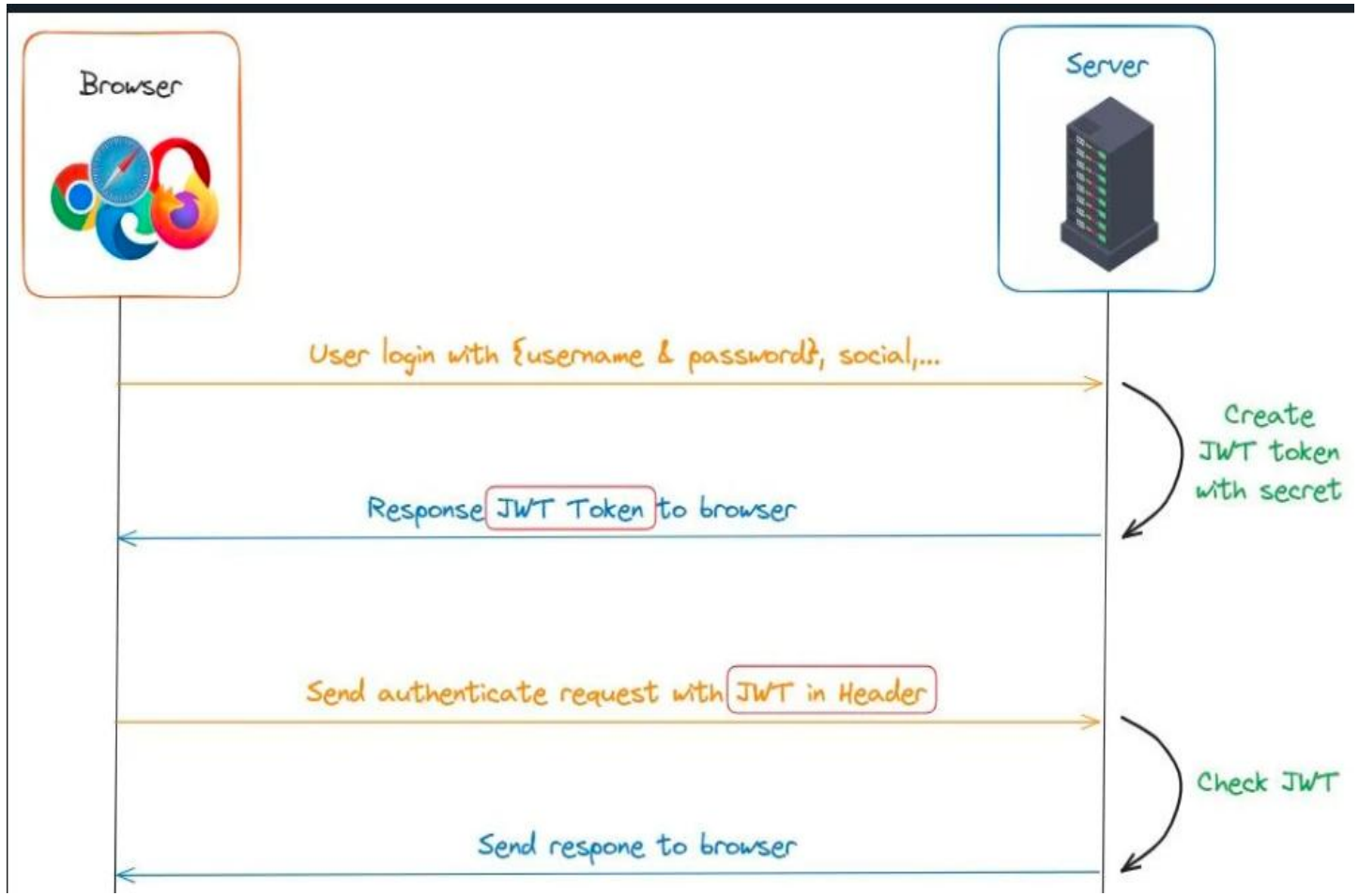
- Trong xác thực, khi người dùng đăng nhập thành công bằng thông tin đăng nhập của họ, JSON Web Token sẽ được trả về. Vì token là thông tin xác thực, cần phải hết sức cẩn thận để ngăn chặn các vấn đề bảo mật. Nói chung, bạn không nên giữ token lâu hơn yêu cầu.
- Bạn cũng không nên lưu trữ dữ liệu nhạy cảm trên session vào trong bộ nhớ trình duyệt do thiếu bảo mật.
- Bất cứ khi nào người dùng muốn truy cập URL hoặc resource được bảo vệ, người dùng nên gửi JWT, thêm Authorization trong header với nội dung là Bearer + token. Nội dung của header như sau: **Authorization: Bearer <token>**



https://jwt.io/introduction/

JSON Web Tokens hoạt động như thế nào?

- Client gửi yêu cầu đăng nhập với thông tin xác thực.
- Server xác thực thông tin và tạo JWT token với secret key, sau đó gửi lại cho client.
- Client sẽ lưu trữ JWT token (thường là trong localStorage, sessionStorage, cookies).
- Từ giai đoạn này, tất cả các request client gửi lên server sẽ đính kèm JWT token.
- Server kiểm tra tính hợp lệ của JWT token bằng cách lấy phần Signature (chữ ký) bên trong token, verify xem chữ ký nhận được chính xác là được hash cùng thuật toán và chuỗi secret key hay không.
- Tùy vào từng trường hợp mà server sẽ phản hồi về cho client.



Lý do sử dụng JWT

- **Authentication** (Xác thực): JWT được sử dụng để xác thực người dùng trước khi họ truy cập đến tài nguyên trên server.
- **Authorization** (Uỷ quyền): Khi người dùng đăng nhập thành công, application có thể truy cập vào các tài nguyên thay mặt người dùng đó. Các ứng dụng đăng nhập một lần (*Single Sign-On SSO*) sử dụng *JWT* thường xuyên vì tính nhỏ gọn và dễ dàng triển khai trên nhiều domain.
- **Trao đổi thông tin an toàn**: *JWT* được coi là một cách trao đổi thông tin an toàn vì thông tin đã được *signed* trước khi gửi đi.

Ưu điểm của JWT

- **Gọn nhẹ**: JWT nhỏ gọn, chi phí truyền tải thấp giúp tăng hiệu suốt của các ứng dụng.
- **Bảo mật**: JWT sử dụng các mật mã khoá để tiến hành xác thực người danh tính người dùng. Ngoài ra, cấu trúc của JWT cho phép chống giả mạo nên thông tin được đảm bảo an toan trong quá trình trao đổi.
- **Phổ thông**: JWT được sử dụng dựa trên JSON, là một dạng dữ liệu phổ biến, có thể sử dụng ở hầu hết các ngôn ngữ lập trình. Ngoài ra, triển khai JWT tương đối dễ dàng và tích hợp được với nhiều thiết bị, vì JWT đã tương đối phổ biến.

Khuyết điểm của JWT

- **Kích thước**: Mặc dù trong tài liệu không ghi cụ thể giới hạn, nhưng do được truyền trên *HTTP Header*, vì thế, *JWT* có giới hạn tương đương với *HTTP Header* (khoảng 8KB).
- **Rủi ro bảo mật**: Khi sử dụng *JWT* không đúng cách, ví dụ như không kiểm tra tính hợp lệ của *signature*, không kiểm tra *expire time*, kẻ tấn công có thể lợi dụng sơ hở để truy cập vào các thông tin trái phép. Ngoài ra, việc để thời gian hết hạn của *JWT* quá dài cũng có thể tạo ra kẽ hở tương tự.

Một số ứng dụng JWT

- **Single Sign-On (SSO)**: *JWT* có thể được sử dụng để cung cấp *single sign-on* cho người dùng. Điều này cho phép họ đăng nhập vào nhiều ứng dụng chỉ với một tài khoản duy nhất.
- **API Authorization**: *JWT* thường được sử dụng để phân quyền cho người dùng đến những tài nguyên cụ thể, từ những *claims* chứa trong JWT đó.
- **User Authentication:** *JWT* cung cấp khả năng xác thực người dùng và cấp quyền cho họ truy cập vào các tài nguyên mong muốn trong hệ thống.
- **Microservices Communication:** *JWT* còn có thể sử dụng cho việc giao tiếp giữa các *service* nhỏ trong hệ thống *microservices*.

Các lỗi khi dùng JWT

- **Sử dụng secret key yếu, dễ dự đoán**: dễ dàng bị tấn công và giải mã dễ dàng.
- **Lưu trữ token không an toàn**: Lưu trữ trong localStorage hay sessionStorage có thể dễ bị tấn công XSS (Cross-Site Scripting).
- **Không thiết lập thời gian hết hạn**: Token sẽ luôn tồn tại vĩnh viễn nếu bạn không thiết lập thời gian hết hạn cho token, nguy cơ về bảo mật nếu như token bị lộ.
- **Truyền qua kết nối không an toàn**: Token dễ bị đánh cắp, luôn sử dụng HTTPs để truyền tải các dữ liệu nhạy cảm.

Lời khuyên khi dùng JWT

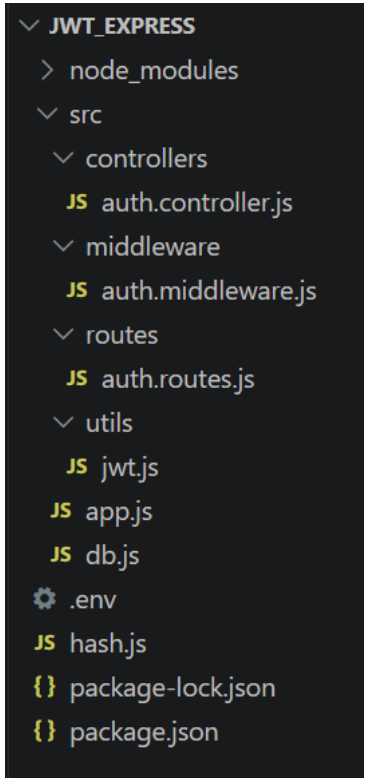
- Cách đang dùng phổ biến hiện nay:
  - Access Token (sống ngắn (5–15 phút)) → lưu **HttpOnly Cookie** hoặc **Memory**
  - Refresh Token (sống dài (7–30 ngày)) → lưu **HttpOnly Cookie** (Database + Rotate) (mỗi lần refresh đổi token mới)
- Flow:
  - Login → server trả access + refresh
  - Access hết hạn → dùng refresh để xin lại
  - Refresh nằm trong cookie → an toàn hơn
  - Logout → xóa cookie

Demo JWT với ExpressJS

- **API Express (login → refresh token → logout)**:
  - JWT + Refresh Token
  - Lưu bằng **HttpOnly Cookie**
  - Có **rotation refresh token**
  - Có middleware bảo vệ route

Demo JWT với ExpressJS

- 1. Cài đặt
  - npm init –y
  - npm install express jsonwebtoken bcryptjs cookie-parser uuid dotenv
- 2. Cấu trúc thư mục



- 3. Config .env
  - ACCESS_TOKEN_SECRET=trung_access_secret_123
    REFRESH_TOKEN_SECRET=trung_refresh_secret_456

Demo JWT với ExpressJS

- 1. Demo DB
  - password: "$2b$10$tyPhQ68pqvny3ZHmE0pdS.Af4.bPYDsd9V4jSQZro.rISz7Spfrgy" // password: 123456

```js
// db.js
const users = [
  {
    id: 1,
    username: "admin",
    password: "$2b$10$tyPhQ68pqvny3ZHmE0pdS.Af4.bPYDsd9V4jSQZro.rISz7Spfrgy" // password: 123456
  }
];

let refreshTokens = [];

module.exports = { users, refreshTokens };
```

- Hash lại password khác nếu cần

```js
const bcrypt = require("bcryptjs");

bcrypt.hash("123456", 10).then(console.log);
```

```
PS D:\jwt_express> node hash.js
$2b$10$tyPhQ68pqvny3ZHmE0pdS.Af4.bPYDsd9V4jSQZro.rISz7Spfrgy
```

Demo JWT với ExpressJS

- 5. JWT Utils

```javascript
// utils/jwt.js
const jwt = require("jsonwebtoken");

const generateAccessToken = (user) => {
  return jwt.sign(
    { id: user.id, username: user.username },
    process.env.ACCESS_TOKEN_SECRET,
    { expiresIn: "15m" }
  );
};

const generateRefreshToken = (user) => {
  return jwt.sign(
    { id: user.id },
    process.env.REFRESH_TOKEN_SECRET,
    { expiresIn: "7d" }
  );
};

module.exports = { generateAccessToken, generateRefreshToken };
```

Demo JWT với ExpressJS

6. Auth Controller

```javascript
// controllers/auth.controller.js
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const { v4: uuidv4 } = require("uuid");
const { users, refreshTokens } = require("../db");
const { generateAccessToken, generateRefreshToken } = require("../utils/jwt");

// LOGIN
exports.login = async (req, res) => {
  const { username, password } = req.body;

  const user = users.find(u => u.username === username);
  if (!user) return res.status(401).json({ message: "User not found" });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ message: "Wrong password" });

  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  refreshTokens.push(refreshToken);

  // set cookie
  res.cookie("accessToken", accessToken, {
    httpOnly: true,
    secure: false, // true nếu HTTPS
    sameSite: "strict",
    maxAge: 15 * 60 * 1000
  });

  res.cookie("refreshToken", refreshToken, {
    httpOnly: true,
    secure: false,
    sameSite: "strict",
    maxAge: 7 * 24 * 60 * 60 * 1000
  });

  res.json({ message: "Login success" });
};
```

```javascript
// REFRESH TOKEN (rotation)
exports.refresh = (req, res) => {
  const token = req.cookies.refreshToken;
  if (!token) return res.sendStatus(401);

  if (!refreshTokens.includes(token)) return res.sendStatus(403);

  jwt.verify(token, process.env.REFRESH_TOKEN_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);

    // remove old token
    const index = refreshTokens.indexOf(token);
    refreshTokens.splice(index, 1);

    const newAccessToken = jwt.sign(
      { id: user.id },
      process.env.ACCESS_TOKEN_SECRET,
      { expiresIn: "15m" }
    );

    const newRefreshToken = jwt.sign(
      { id: user.id },
      process.env.REFRESH_TOKEN_SECRET,
      { expiresIn: "7d" }
    );

    refreshTokens.push(newRefreshToken);

    res.cookie("accessToken", newAccessToken, {
      httpOnly: true,
      sameSite: "strict"
    });

    res.cookie("refreshToken", newRefreshToken, {
      httpOnly: true,
      sameSite: "strict"
    });

    res.json({ message: "Refreshed" });
  });
```

```javascript
};

// LOGOUT
exports.logout = (req, res) => {
  const token = req.cookies.refreshToken;

  if (token) {
    const index = refreshTokens.indexOf(token);
    if (index > -1) refreshTokens.splice(index, 1);
  }

  res.clearCookie("accessToken");
  res.clearCookie("refreshToken");

  res.json({ message: "Logged out" });
};
```

7. Middleware bảo vệ API

```
// middleware/auth.middleware.js
const jwt = require("jsonwebtoken");

exports.verifyToken = (req, res, next) => {
  const token = req.cookies.accessToken;
  if (!token) return res.sendStatus(401);

  jwt.verify(token, process.env.ACCESS_TOKEN_SECRET, (err, user) => {
    if (err) return res.sendStatus(403);
    req.user = user;
    next();
  });
};
```

Demo JWT với ExpressJS

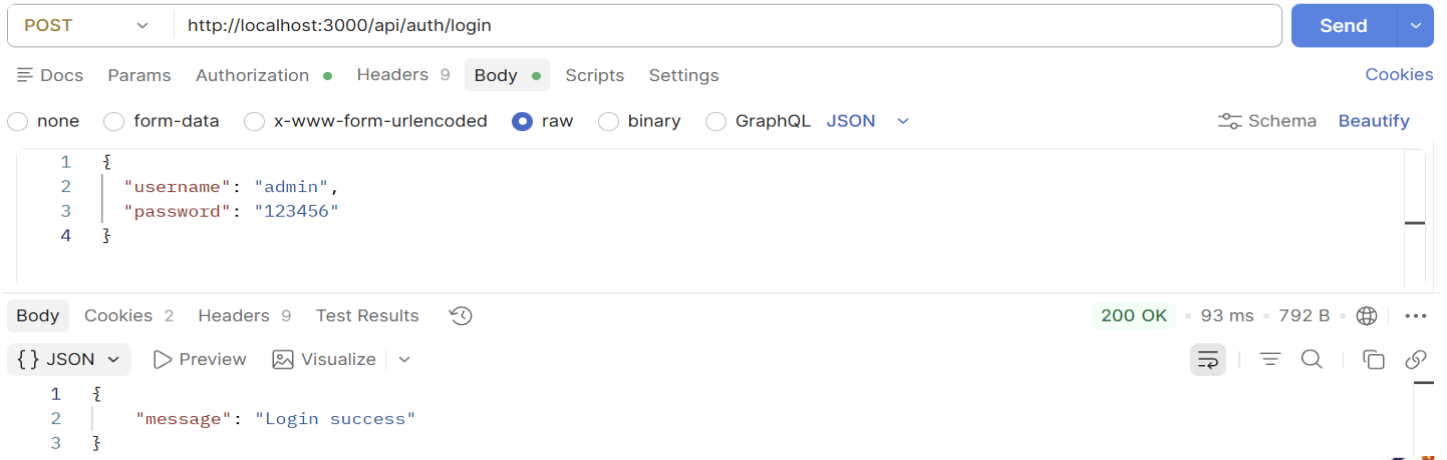
8. Routes

```javascript
// routes/auth.routes.js
const express = require("express");
const router = express.Router();
const authController = require("../controllers/auth.controller");
const { verifyToken } = require("../middleware/auth.middleware");
const rateLimit = require("express-rate-limit");

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 phút
  max: 5, // 5 lần thử
  message: "Too many login attempts"
});

router.post("/login", loginLimiter, authController.login);
router.post("/refresh", authController.refresh);
router.post("/logout", authController.logout);

// protected route
router.get("/profile", verifyToken, (req, res) => {
  res.json({ user: req.user });
});
router.get("/api/csrf-token", (req, res) => {
  res.json({ csrfToken: req.csrfToken() });
});
module.exports = router;
```

9. App.js

```javascript
const express = require("express");
const cookieParser = require("cookie-parser");
require("dotenv").config();

const app = express();
const csurf = require("csurf");

const csrfProtection = csurf({ cookie: true });

app.use(csrfProtection);

app.use(express.json());
app.use(cookieParser());

const authRoutes = require("./routes/auth.routes");
app.use("/api/auth", authRoutes);

app.listen(3000, () => console.log("Server running on port 3000"));
```

Test trên postman



10. Chạy project

node src/app.js

Demo JWT với ExpressJS

- Test trên postman

POST http://localhost:3000/api/auth/login Send

Docs Params Authorization Headers 9 Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "username": "admin",
3   "password": "123456"
4 }
```

Body Cookies 2 Headers 9 Test Results 200 OK 93 ms 792 B

{} JSON Preview Visualize

```
1 {
2   "message": "Login success"
3 }
```

Body Cookies 2 Headers 9 Test Results 200 OK 93 ms 792 B

Key	Value
X-Powered-By	Express
Set-Cookie	accessToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm...
Set-Cookie	refreshToken=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiaWF0Ijo...
Content-Type	application/json; charset=utf-8
Content-Length	27
ETag	W/"1b-r6HkcREyQaRHbryVd7hGUkeRtJM"
Date	Mon, 04 May 2026 07:18:24 GMT
Connection	keep-alive
Keep-Alive	timeout=5

Demo JWT với ExpressJS

- Test trên postman

POST http://localhost:3000/api/auth/refresh Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
Key	Value	Description	

Body Cookies 2 Headers 9 Test Results 200 OK 17 ms 655 B

Name	Value	Domain	Path	Expires	HttpOnly	Secure
accessToken	eyJhbGciOiJIUzI1NiIsInR5c...	localhost	/	Session	true	false
refreshToken	eyJhbGciOiJIUzI1NiIsInR5c...	localhost	/	Session	true	false

GET http://localhost:3000/api/auth/profile Send

Docs Params Authorization Headers 8 Body Scripts Settings Cookies

Auth Type: Bearer Token

Token: PUhasLHxbtElQSDjZrE7_965hOU5yJ6s

The authorization header will be

Body Cookies 2 Headers 7 Test Results 200 OK 3 ms 286 B

Name	Value	Domain	Path	Expires	HttpOnly	Secure
accessToken	eyJhbGciOiJIUzI1...	localhost	/	Session	true	false
refreshToken	eyJhbGciOiJIUzI1...	localhost	/	Session	true	false

Demo JWT với ExpressJS

- Test trên postman
  - Logout sẽ xóa accessToken và refreshToken và sẽ không truy cập được profile

POST http://localhost:3000/api/auth/logout Send

Docs Params Authorization Headers 7 Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit ···
Key	Value	Description	

Body Cookies Headers 9 Test Results 200 OK • 12 ms • 406 B •••

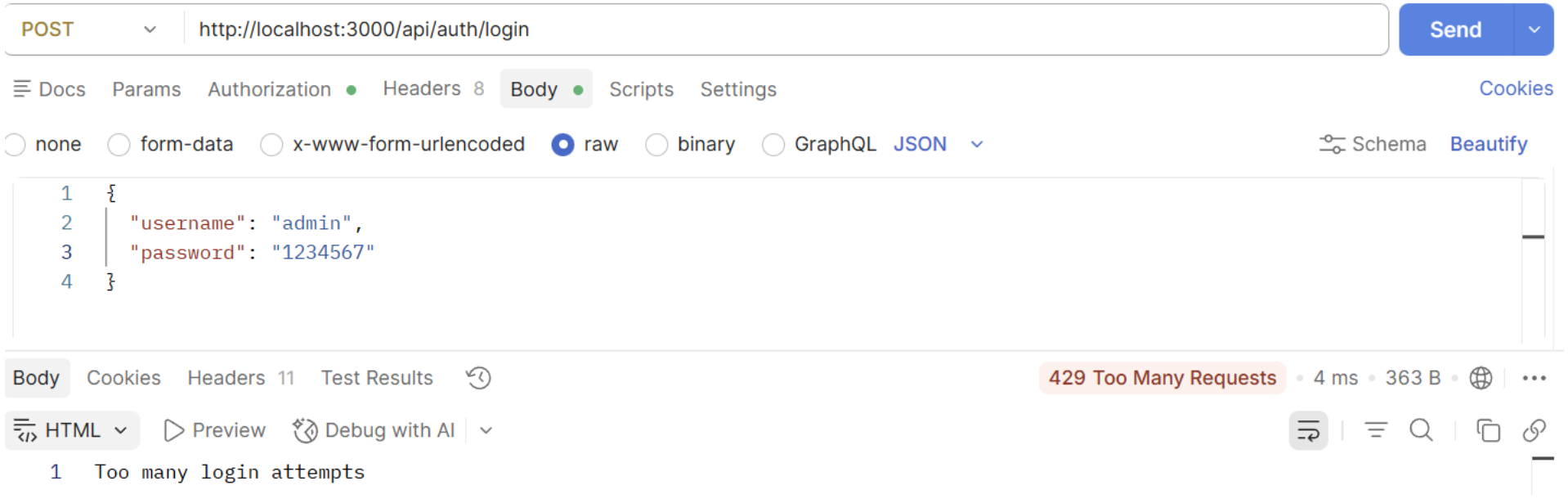
Key	Value
X-Powered-By	Express
Set-Cookie	accessToken=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT
Set-Cookie	refreshToken=; Path=/; Expires=Thu, 01 Jan 1970 00:00:00 GMT
Content-Type	application/json; charset=utf-8
Content-Length	24
ETag	W/"18-ez3HU6ztP+x97ArcQd0fxpuaOt0"
Date	Mon, 04 May 2026 07:25:18 GMT
Connection	keep-alive
Keep-Alive	timeout=5

Demo JWT với ExpressJS

- Test Rate Limiting
  - Thiết lập ratelimit trong routes.js.

```
6   const rateLimit = require("express-rate-limit");
7
8   const loginLimiter = rateLimit({
9     windowMs: 15 * 60 * 1000, // 15 phút
10    max: 5, // 5 lần thử
11    message: "Too many login attempts"
12  });
13
14  router.post("/login", loginLimiter, authController.login);
```

  - Rồi nhập mật khẩu sai 6 lần sẽ thông báo lỗi



Lưu token vào Database Redis

- secure: true khi deploy HTTPS
- Lưu refreshToken trong DB (Redis tốt nhất)
- Hash refreshToken trước khi lưu

Lưu token vào Database Redis

- 1. secure: true khi dùng HTTPS

```
res.cookie("accessToken", accessToken, {
  httpOnly: true,
  secure: true,        // 🔥 bắt buộc khi deploy HTTPS
  sameSite: "strict",
  maxAge: 15 * 60 * 1000
});
```

- Nếu deploy sau reverse proxy (Nginx, Cloudflare):
  - app.set("trust proxy", 1);

Lưu token vào Database Redis

- 2. Lưu Refresh Token bằng Redis
  - npm install redis

```
// src/config/redis.js
const { createClient } = require("redis");

const redisClient = createClient({
  url: "redis://127.0.0.1:6379"
});

redisClient.on("error", (err) => console.error("Redis error:", err));

(async () => {
  await redisClient.connect();
  console.log("✅ Redis connected");
})();

module.exports = redisClient;
```

- 3. Hash Refresh Token trước khi lưu: npm install crypto

```
// utils/hash.js
const crypto = require("crypto");

const hashToken = (token) => {
  return crypto.createHash("sha256").update(token).digest("hex");
};

module.exports = { hashToken };
```

Lưu token vào Database Redis

- 4. Login (lưu Redis + hash)

```
// LOGIN
exports.login = async (req, res) => {
  const { username, password } = req.body;

  const user = users.find(u => u.username === username);
  if (!user) return res.status(401).json({ message: "User not found" });

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(401).json({ message: "Wrong password" });

  const accessToken = generateAccessToken(user);
  const refreshToken = generateRefreshToken(user);

  // 🔥 THÊM ĐOẠN NÀY
  const hashed = hashToken(refreshToken);

  await redis.set(
    `refresh:${user.id}`,
    hashed,
    { EX: 7 * 24 * 60 * 60 }
  );

  console.log("✅ Saved refresh token:", `refresh:${user.id}`);

  // set cookie
  res.cookie("accessToken", accessToken, {
    httpOnly: true,
    secure: false,
    sameSite: "strict",
    maxAge: 15 * 60 * 1000
  });

  res.cookie("refreshToken", refreshToken, {
    httpOnly: true,
    secure: false,
    sameSite: "strict",
    maxAge: 7 * 24 * 60 * 60 * 1000
  });

  res.json({ message: "Login success" });
};
```

- 5. Refresh Token (rotation + chống reuse attack)

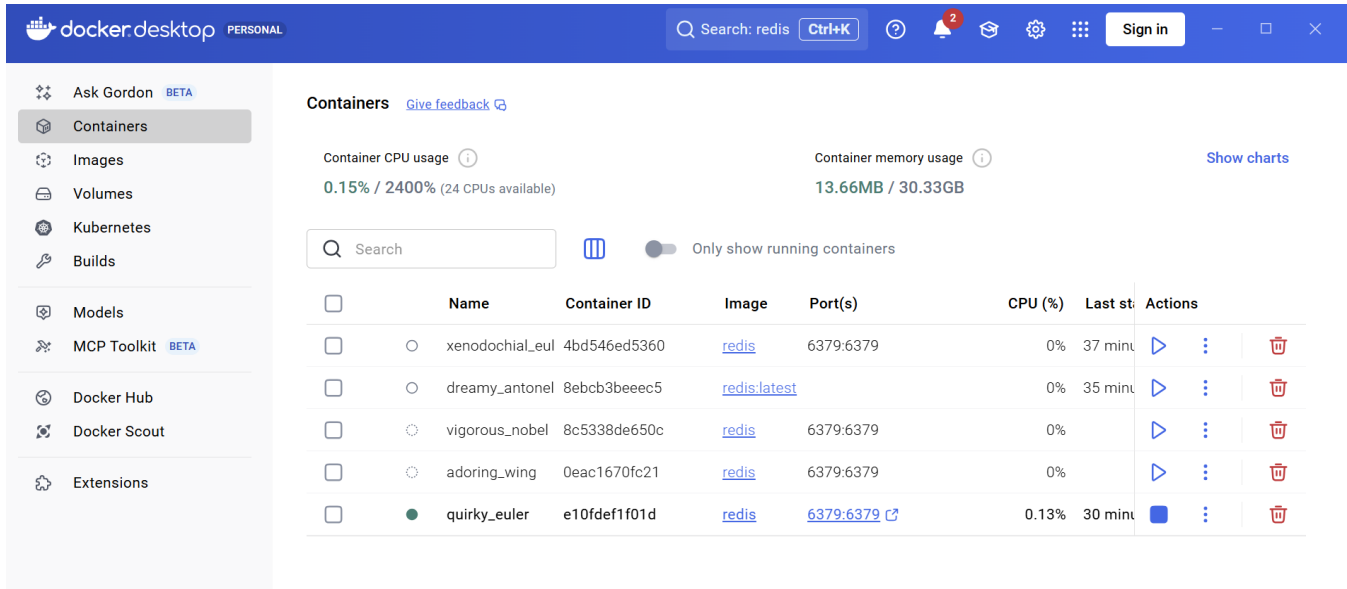
```
exports.refresh = async (req, res) => {
  const token = req.cookies.refreshToken;
  if (!token) return res.sendStatus(401);

  const hashed = hashToken(token);

  const decoded = jwt.verify(token, process.env.REFRESH_TOKEN_SECRET);

  const stored = await redis.get(`refresh:${decoded.id}`);

  // ❌ token không khớp -> có thể bị hack
  if (!stored || stored !== hashed) {
    return res.sendStatus(403);
  }

  // 🔥 rotation: xóa token cũ
  await redis.del(`refresh:${decoded.id}`);

  const newRefreshToken = generateRefreshToken({ id: decoded.id });
  const newHashed = hashToken(newRefreshToken);

  await redis.set(
    `refresh:${decoded.id}`,
    newHashed,
    { EX: 7 * 24 * 60 * 60 }
  );

  const newAccessToken = generateAccessToken({ id: decoded.id });

  res.cookie("accessToken", newAccessToken, {
    httpOnly: true,
    secure: true,
    sameSite: "strict"
  });

  res.cookie("refreshToken", newRefreshToken, {
    httpOnly: true,
    secure: true,
    sameSite: "strict"
  });

  res.json({ message: "Refreshed" });
};
```

Lưu token vào Database Redis

- 6. Logout (xóa Redis)

```
150 exports.logout = async (req, res) => {
151   const token = req.cookies.refreshToken;
152
153   if (token) {
154     const decoded = jwt.decode(token);
155     if (decoded?.id) {
156       await redis.del(`refresh:${decoded.id}`);
157     }
158   }
159
160   res.clearCookie("accessToken");
161   res.clearCookie("refreshToken");
162
163   res.json({ message: "Logged out" });
164 };
```

- 7. Test Redis: Dùng Docker để dựng 01 server redis



Lưu token vào Database Redis

- 7. Test Redis:
  - Start server redis
  - Vào cmd gõ: docker exec -it <container-id> redis-cli
  - Vào Docker lấy container-id dán vào lệnh trên.
  - Chạy postman để chạy api login để sinh accessToken và refreshToken.
  - Gõ: get refresh:1 để kiểm tra mã hash lưu trữ trong redis

```
127.0.0.1:6379> get refresh:1
"e9254f1757aa31639c38cd25cc80d6078c01abcfdea79df16344b04281220312"
127.0.0.1:6379>
```